

Task 2 Part C: Counting Bloom Filter Analysis

Issues with Deletion in Counting Bloom Filters

The main issues with deletion in counting Bloom filters arise from the probabilistic nature of the data structure and the shared counters:

1. **False Negatives:** When deleting an item that shares bucket counters with other items, we might accidentally decrease counters that are also being used by other items, potentially creating false negatives. This violates the Bloom filter's guarantee of never having false negatives.
2. **Counter Limitations:** With c -bit counters, we can only count up to $2^c - 1$ items per bucket. This creates an upper bound on how many items can share the same bucket, potentially leading to overflow situations when the filter becomes too full.
3. **Deletion Ambiguity:** Since multiple items can share the same bucket counters, we cannot be certain that decreasing a counter won't affect other items that legitimately exist in the filter.

Complexity Analysis

Let:

- n = number of items in the filter
- k = number of hash functions
- m = number of buckets
- c = number of bits per counter (variable)

Space Complexity

- Standard Bloom Filter: $O(m)$ bits
- Counting Bloom Filter: $O(m * c)$ bits
 - Each bucket now requires c bits instead of 1 bit
 - Total space increased by factor of c

Time Complexity

- Insertion: $O(k)$ operations
 - Must compute k hashes and update k counters
 - Each counter update now requires $O(c)$ bit operations
 - Total: $O(k * c)$
- Deletion: $O(k * c)$
 - Must compute k hashes and update k counters
 - Each counter update requires $O(c)$ bit operations
 - Additional checks for underflow
- Query: $O(k)$

- Same as standard Bloom filter
- Reading c -bit counters doesn't significantly impact asymptotic complexity

Overall Impact

The modification from Part A to Part B introduces a factor of c to both space complexity and update operations (insert/delete), while maintaining the same query complexity. The practical impact depends on the chosen value of c , with $c = 4$ being common but potentially insufficient for some applications requiring higher counts.

Task 2 Part E: Dynamic Bloom Filter Analysis

The transition from a basic Bloom filter (Part A) to a dynamic Bloom filter (Part D) introduces significant complexity changes to handle unlimited growth while maintaining efficient operations. While Part A's basic Bloom filter has fixed $O(m)$ space complexity and $O(k)$ operations, our dynamic implementation scales with the number of filters s while optimizing for practical performance.

Our space complexity increases from $O(m)$ to $O(s * m * c)$, where s is the number of filters, m is buckets per filter, and c is bits per counter. However, this allows for unlimited growth compared to Part A's fixed-size limitation. For time complexity, insertion remains $O(k)$ as we only insert into the latest filter, but queries become $O(k * s)$ worst case since we may need to check all filters. We achieve $O(k + s)$ deletion by:

1. Using `birdNames` arrays to track filter membership in $O(s)$ time
2. Maintaining `first_bird/last_bird` pointers to quickly determine relevant filters
3. Only processing unique hash values to reduce redundant operations

Similar to hash tables where we consider load factor constant, we treat `BUCKET_SIZE` (c) and `NUM_HASHES` (k) as constants. This makes our space-time tradeoff worthwhile: we get unlimited capacity and efficient deletion at the cost of some space overhead for tracking metadata.